200. Number of Islands Medium

Acceptance: 63.6% | LeetCode

Given an `m x n` 2D binary grid `grid` which represents a map of `'1'`s (land) and `'0'`s (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
**Input:** grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
**Output:** 1
```

Example 2:

```
**Input:** grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
**Output:** 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is `'0'` or `'1'`.

Claude Opus Solution

Number of Islands

1. Intuition

The key insight is that an **island is a connected component** of `'1'`s in the grid. Two land cells belong to the same island if they're connected horizontally or vertically.

This is a classic **flood fill / connected components** problem on a grid (treating the grid as an implicit graph where each cell is a node connected to its 4 neighbors).

The trick: Every time we encounter an unvisited piece of land, it must be part of a *new* island. We then "flood" the entire island (mark all connected land as visited) so we don't count it again. The number of times we initiate a flood = the number of islands.

2. Approach (Pseudocode)

```
count = 0
for each cell (i, j) in grid:
    if cell is land ('1'):
        count += 1                  # found a new island
        flood_fill(i, j)            # sink the entire island

flood_fill(r, c):
    if out of bounds OR cell is water ('0'):
        return
    mark cell as visited (set to '0')
    recurse on 4 neighbors (up, down, left, right)

return count
```

3. Python Solution

Solution A: DFS (Recursive)

```python
from typing import List


class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
```

```
            return 0

        rows, cols = len(grid), len(grid[0])

        def dfs(r: int, c: int) -> None:
            # Base case: stop if out of bounds OR we hit water/visited cell
            if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == "0":
                return

            # Mark current land cell as visited by "sinking" it to water.
            # This prevents revisiting and double-counting.
            grid[r][c] = "0"

            # Explore all 4 directions to flood the connected land.
            dfs(r - 1, c)  # up
            dfs(r + 1, c)  # down
            dfs(r, c - 1)  # left
            dfs(r, c + 1)  # right

        islands = 0
        # Scan every cell in the grid.
        for i in range(rows):
            for j in range(cols):
                # Each unvisited land cell starts a brand-new island.
                if grid[i][j] == "1":
                    islands += 1   # count this new island
                    dfs(i, j)      # sink all land connected to it

        return islands
```

Solution B: BFS (Iterative — safer for large grids)

Recursion can hit Python's recursion limit (~1000) on a 300×300 grid with one giant island (up to 90,000 deep). **BFS avoids stack overflow.**

```
from typing import List
from collections import deque


class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        islands = 0

        def bfs(start_r: int, start_c: int) -> None:
            queue = deque([(start_r, start_c)])
            grid[start_r][start_c] = "0"  # mark visited immediately

            while queue:
                r, c = queue.popleft()
                # Check all 4 neighbors
                for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                    nr, nc = r + dr, c + dc
                    # If neighbor is in-bounds and is land, add to queue
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == "1":
                        grid[nr][nc] = "0"  # mark BEFORE enqueuing to avoid duplicates
                        queue.append((nr, nc))

        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == "1":
                    islands += 1
                    bfs(i, j)

        return islands
```

---

4. Complexity Analysis

Let `m = rows`, `n = cols`.

Approach	Time	Space
DFS	**O(m × n)**	**O(m × n)** worst case
BFS	**O(m × n)**	**O(min(m, n))** typical, O(m × n) worst

Time — O(m × n): We visit each cell at most twice — once in the outer scan loop, and once during a flood fill (after which it's marked `'0'` and never re-explored). Linear in the number of cells.

Space: - **DFS:** The recursion call stack can be as deep as the number of cells in one island. Worst case (entire grid is one island) = **O(m × n)**. - **BFS:** The queue holds at most the "frontier" of the flood. In the worst case this can also reach O(m × n), but it's often more bounded (~O(min(m, n))) and importantly **doesn't risk a recursion-limit crash**.

> **Note:** Both modify the input grid (mutating `'1'` → `'0'`). If you must preserve the input, use a separate `visited` set/matrix at the cost of extra O(m × n) space.

---

5. Alternative Approaches

1. **Union-Find (Disjoint Set Union):**
2. Treat each land cell as a node. Union adjacent land cells.
3. The number of islands = number of distinct connected components (roots).
4. Time: O(m × n × α(m×n)) ≈ near-linear with path compression + union by rank.
5. Useful when the grid is **dynamic** (e.g., LeetCode 305 "Number of Islands II" where land is added over time).
6. **Iterative DFS with explicit stack:**
7. Same logic as recursive DFS but uses a manual stack — combines DFS traversal with crash-safety from avoiding deep recursion.
8. **Non-destructive variant:**
9. Use a `visited` boolean matrix instead of mutating `grid`. Same complexity, preserves input.

Recommendation: For interviews, present the **DFS solution** for its clarity, but **mention BFS** to show awareness of recursion-depth limits on large inputs (the 300×300 constraint here makes this a real concern!).

Codex Solution

1. Intuition

Treat the grid as a **graph**:

- Each land cell `'1'` is a node.
- Two land cells are connected if they are adjacent **up, down, left, or right**.
- An island is one connected component of land cells.

So the problem becomes:

> Count the number of connected components made of `'1'` cells.

Whenever we find an unvisited land cell, we have discovered a **new island**. Then we explore all land cells connected to it using DFS or BFS and mark them as visited so we do not count the same island again.

A common trick is to mark visited land cells by changing them from `'1'` to `'0'`.

---

2. Approach

We can use DFS.

Pseudocode

```
function numIslands(grid):
    rows = number of rows in grid
    cols = number of columns in grid

    islands = 0

    for each row r:
        for each column c:
            if grid[r][c] == '1':
                islands += 1

                perform DFS from (r, c):
                    mark every connected land cell as visited
                    visited cells can be changed to '0'

    return islands
```

DFS Pseudocode

```
function dfs(start_row, start_col):
    stack = [(start_row, start_col)]

    while stack is not empty:
        row, col = stack.pop()

        if row or col is out of bounds:
            continue

        if grid[row][col] == '0':
            continue

        mark grid[row][col] as '0'

        add its 4 neighbors to stack:
            row - 1, col
            row + 1, col
            row, col - 1
            row, col + 1
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        # Number of rows and columns in the grid
        rows = len(grid)
        cols = len(grid[0])

        # Counter for the number of islands found
        islands = 0

        # Four possible directions:
        # up, down, left, right
        directions = [
            (-1, 0),  # up
            (1, 0),   # down
            (0, -1),  # left
            (0, 1)    # right
        ]

        def dfs(start_row: int, start_col: int) -> None:
            """
            Iterative DFS that visits all land cells connected to
            grid[start_row][start_col].

            We mark visited land cells as '0' so they are not counted again.
            """
```

```
            # Stack stores cells we still need to process
            stack = [(start_row, start_col)]

            while stack:
                row, col = stack.pop()

                # If the cell is out of bounds, ignore it
                if row < 0 or row >= rows or col < 0 or col >= cols:
                    continue

                # If the cell is water or already visited, ignore it
                if grid[row][col] == "0":
                    continue

                # Mark this land cell as visited by turning it into water
                grid[row][col] = "0"

                # Add all 4 neighboring cells to the stack
                for dr, dc in directions:
                    new_row = row + dr
                    new_col = col + dc
                    stack.append((new_row, new_col))

        # Scan every cell in the grid
        for r in range(rows):
            for c in range(cols):

                # If we find land, this is the start of a new island
                if grid[r][c] == "1":
                    islands += 1

                    # Visit and mark the entire island
                    dfs(r, c)

        return islands
```

---

4. Complexity Analysis

Let:

- `m` be the number of rows
- `n` be the number of columns

Time Complexity: `O(m * n)`

We visit each cell at most once.

Even though DFS checks neighboring cells, each land cell is marked as water after being visited, so it will not be processed again as part of another island.

Therefore, total time is:

```
O(m * n)
```

Space Complexity: `O(m * n)`

The stack used by DFS can contain up to all land cells in the worst case.

For example, if the entire grid is land, the DFS may store many cells in the stack.

So the worst-case space complexity is:

```
O(m * n)
```

The solution modifies the input grid in-place, so no extra visited matrix is needed.

---

5. Alternative Approaches

1. Recursive DFS

Instead of using an explicit stack, we can use recursion:

```
def dfs(r, c):
    if r < 0 or r >= rows or c < 0 or c >= cols:
        return
    if grid[r][c] == "0":
        return

    grid[r][c] = "0"

    dfs(r - 1, c)
    dfs(r + 1, c)
    dfs(r, c - 1)
    dfs(r, c + 1)
```

This is simpler, but in Python it may hit recursion depth limits for large grids.

---

2. BFS

Use a queue instead of a stack.

The idea is the same:

- Find a land cell.
- Count a new island.
- Use BFS to visit all connected land cells.
- Mark them as visited.

Time and space complexity are also `O(m * n)`.

---

3. Union-Find / Disjoint Set Union

We can treat every land cell as a node and union adjacent land cells.

After processing the grid, the number of connected components is the number of islands.

This is useful when there are many dynamic connectivity operations, but for this static problem, DFS or BFS is simpler.